Lab task 5

NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Name: Muhammad Musawar Ali Shah

Roll No. 24P-0619

Dept: BS-CS

Section: BCS-3D

Subject: Coal Lab

Submitted To: Sir Hasaan Shaukat

Lab 5

Question 1:

Code	Explanation

[org 0x0100]
mov ax, 0
mov bx, 0
mov cx, 7
start:
  add ax, [data + bx]
  add bx, 2
  dec cx
  jnz start
mov [result], ax
mov ax, 0x4c00
int 0x21
data: dw 10, 20, 30, 40, 50, 60, 70
result: dw 0
	❖ Initialized ax and bx register with 0.
	❖ CX register starts at 7 to execute 7 iterations.
	❖ At line 5, there is a label **start** which starts the loop.
	❖ In loop body, I used indirect addressing to add data into ax register.
	❖ Decreased cx value by 1.
	❖ Used jump statement to check the condition (if cx not 0 then goto start:).
	❖ At the end store the final result in result label.

Question 2:

Code	Explanation

[org 0x0100]
mov ax, 0
mov bx, 0
start:
  add ax, [data + bx]
  add bx, 2
  cmp bx, 14
  jne start
mov [result], ax
mov ax, 0x4c00
int 0x21
data: dw 10, 20, 30, 40, 50, 60, 70
result: dw 0
	❖ At line 2,3 I initialized ax and bx register with 0 value.
	❖ Line 3, loop started.
	❖ Adding values in ax registers one by one with the helo of indirect addressing [data + bx].
	❖ Adding value to bx (moving to next address).
	❖ Check if value of bx equal to 14.
	❖ Jump if not equal means if (bx = 14) then stop.
	❖ Store final result into the result and then terminating instructions

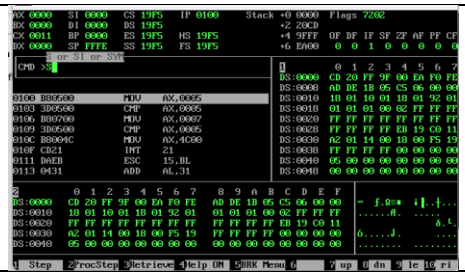
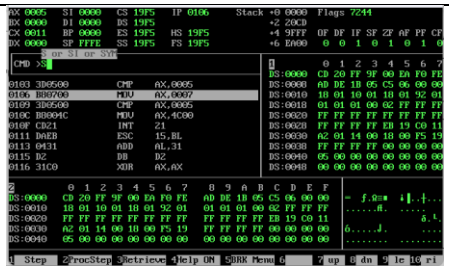
Question 3:

Code	Explanation

[org 0x0100]
mov bx, 0
mov ax, [data + bx]
add bx, 2
mov cx, 6
start:
  cmp ax, [data + bx]
  jge skip
  mov ax, [data + bx]
skip:
  add bx, 2
  dec cx
  jnz loop_start
mov [result], ax
mov ax, 0x4c00
int 0x21
 	
data: dw 10, 85, 30, 40, 95, 60, 70
result: dw 0
	❖ Line 2, initialized bx as 0
	❖ Mov ax, [data + bx] loading first element as a initial largest number (assuming that it is largest numbe).
	❖ Line 4, adding 2 in bx. It help in moving through all the elements of the array.
	❖ Counter 6, to check conditions.
	❖ Line 7, compare the current number with the next number.
	❖ Jump to next label **skip** if curr value is grater than next value and if not update current value with the next value.
	❖ Add 2 into bx register to move to the next element.
	❖ Decrease counter and check if the value of cx is equal to 0 if it is then stop.
	❖ Mov the final result into result
	❖ Terminate the program

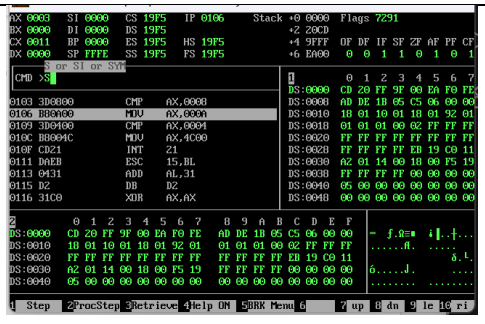
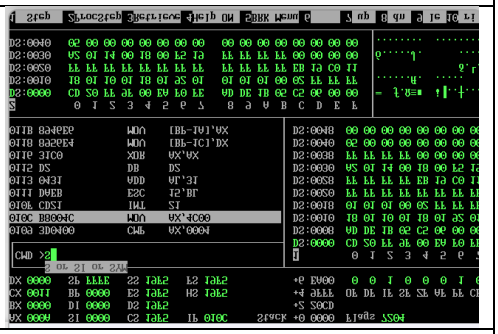
Question 4:

Code Zero Flag	When ZF = 0	When ZF = 1

[org 0x0100]
mov ax, 5
cmp ax, 5
mov ax, 7
cmp ax, 5
mov ax, 0x4c00
int 0x21
		

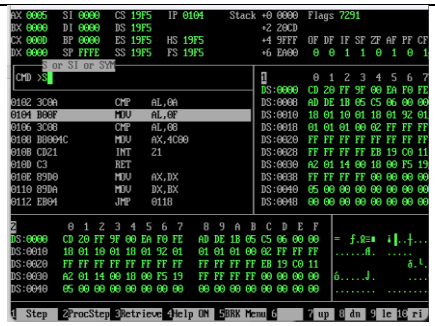
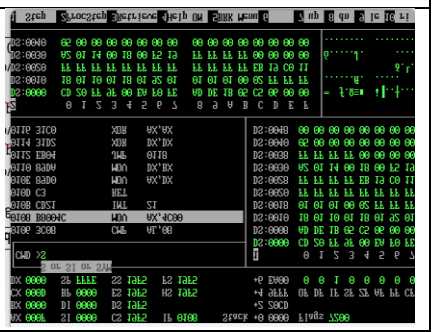
	Initially ZF = 0	When reached cmp ax, 5 result is 0 then ZF = 1

Code Sign Flag	SF = 1	SF = 0

[org 0x0100]
 	
mov ax, 3
cmp ax, 8
mov ax, 10
cmp ax, 4
 	
mov ax, 0x4c00
int 0x21
		

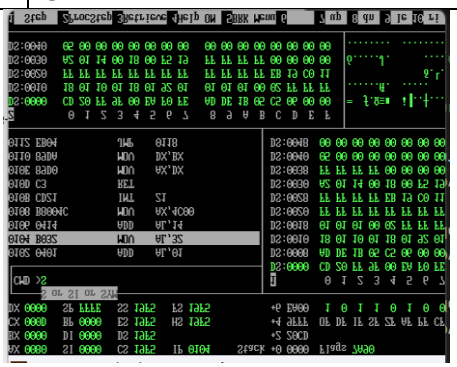
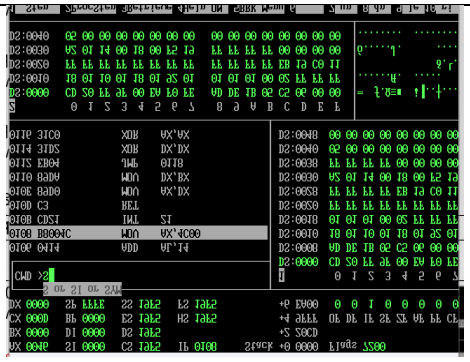
	Cmp ax, 8 (3-8 = -5) then SF = 1	Cmp ax,4 (10-4) then SF = 0

Code Caryy Flag	CF = 1	CF = 0

[org 0x0100]
 	
mov al, 5
cmp al, 10
mov al, 15
cmp al, 8
 	
mov ax, 0x4c00
int 0x21
		

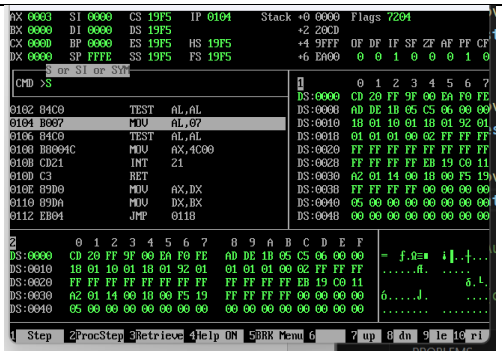
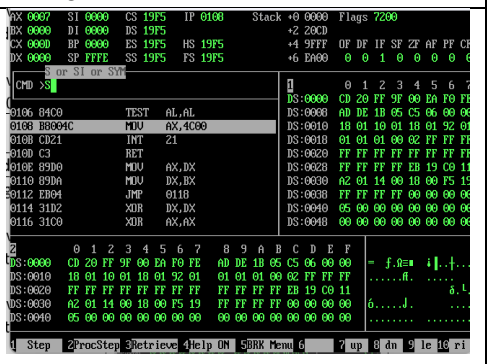
	Cmp al, 10 (5-10) CF = 1	Cmp al, 8 (15-8) CF = 0

Code Overflow Flag	OF = 1	OF = 0

[org 0x0100]
 	
mov al, 127
add al, 1
 	
mov al, 50
add al, 20
 	
mov ax, 0x4c00
int 0x21
		

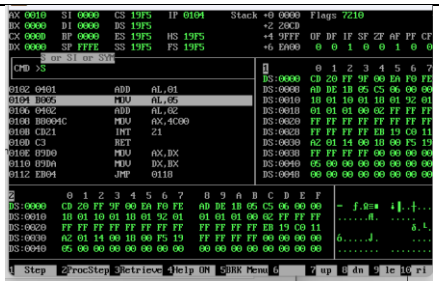
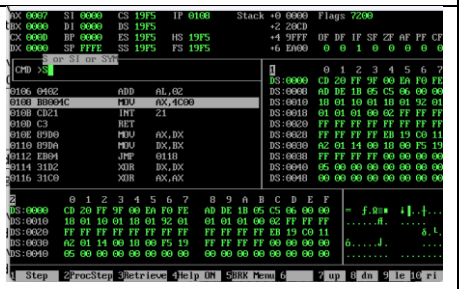
	Add al,1 (127 + 1) OF = 1	Add al 20 (20+50) then OF = 0

Code Parity Flag	PF = 1	PF = 0

[org 0x0100]
 	
mov al, 3
test al, al
 	
mov al, 7
test al, al
 	
mov ax, 0x4c00
int 0x21
		

	Mov al, 4 (00000011) 2 ones means PF = 1	Mov al, 7 (00000111) 4 ones PF = 0

Code auxiliary Flag	AF = 1	AF = 0

[org 0x0100]
 	
mov al, 0x0F
add al, 1
mov al, 5
add al, 2
mov ax, 0x4c00
int 0x21
		

	Add al, 1(carry from low to high nibble AF = 1	Add al, 2 no carry between nibbles AF = 0

Question 5:

Code	Explanation

[org 0x0100]
mov bx, 0
mov cx, 5
loop:
  mov ax, [data + bx]
  cmp ax, 0
  je store_zero
  jmp check_properties
store_zero:
  mov word [results + bx], 0
  jmp next_number
check_properties:
  cmp ax, 0
  jl neg:
  mov word [results + bx], 1
  jmp next_number
neg:
  mov word [results + bx], 2
next_number:
  add bx, 2
  dec cx
  jnz loop
mov ax, 0x4c00
int 0x21
data: dw 0, -5, 10, -3, 7
results: dw 0, 0, 0, 0, 0
	❖ At line 2,3 moved 0 in bx and 5 in cx.
	❖ Line 4, labeled loop.
	❖ Line 5, load first number.
	❖ Line 6, check if the number is 0
	❖ If 0 then store the value and continue.
	❖ If not zero then jump into label “check_properties”.
	❖ Line 13, Check if the number is negative
	❖ If it is then jump to label “neg”
	❖ Line 16, If it is positive then store 1 for positive and jump to next number.
	❖ Line 18, If the number is negative then store 2.
	❖ Move to next number.
	❖ Decreament counter register.
	❖ Continue the procedure until counter = 0.